PRÁCTICA 1
Procesos, hilos y tuberías

Fernando Blanco Varela (*fernando.blancov@estudiante.uam.es*)

Danyyil Shykerynets (*danyyil.shykerynets@estudiante.uam.es*)

Grupo 2271

19-02-2026

Escuela
Politécnica Superior

1. INTRODUCCIÓN

En esta primera práctica, nos familiarizamos con la programación de bajo nivel. Manejamos procesos y creación de procesos hijos, previniendo huérfanos y zombies; dividimos la carga de trabajo en hilos y estudiamos la eficiencia de ello; y realizamos comunicación entre procesos (IPC) mediante tuberías.

2. CUMPLIMIENTO DE REQUISITOS

Hemos cumplido todos los requisitos que se han pedido:

- Ejecución del sistema con los parámetros indicados.
- Sistema multiproceso: El minero y el registrador son dos procesos, siendo el registrador hijo del minero. El manejo de ambos se realiza desde un main.
- Minero multihilo: El minero divide correctamente la carga de trabajo en hilos, y separa los rangos de búsqueda entre los hilos.
- Registrador del sistema: Los procesos se comunican correctamente a través de las tuberías y el registro es acorde al que se pide.
- Pruebas exhaustivas y control de errores.
- Análisis del rendimiento.
- Documentación con Doxygen.

3. PRUEBAS REALIZADAS

3.1. Prueba trivial

En primer lugar, verificamos que los resultados que obtenemos son acordes a los del enunciado de la práctica.

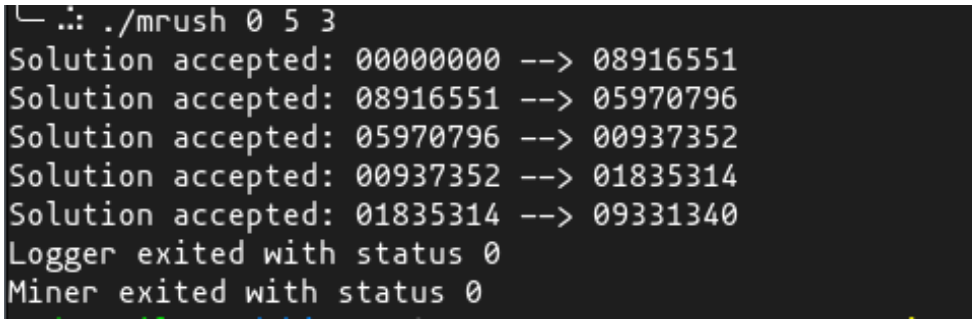

Figura 1: Prueba del enunciado

3.2. Parámetros de entrada generales

En primer lugar, hemos implementado un control de errores exhaustivo mediante la función assert() de la librería assert.h. Con esto nos aseguramos en la mayoría de las funciones que los parámetros sean los necesarios (en los que no se usa assert, se devuelve un código de error). Por ejemplo la función logger en logger_pipe.c:

```
i32 logger(i32 *miner_pipe, i32 *logger_pipe) {
  /* VERIFICACION PARAMETROS DE ENTRADA */
  assert(miner_pipe != NULL);
  assert(logger_pipe != NULL);

...

}
```

También se ha implementado una macro de control de error para retornar mensajes de error dados por errno y salir del programa:

```
/**
 * @brief Imprimir error y terminar el programa
 *
 * @param msg Cadena con mensaje de error
 */
#define die(msg)                                                    \
  do {                                                              \
    perror(msg);                                                    \
    exit(EXIT_FAILURE);                                             \
  } while (0)
```

```
//EJEMPLO DE USO
u64 *pow_result = (u64 *)malloc(sizeof(u64));
if (pow_result == NULL)
  die(strerror(errno));
```

3.3. Parámetros del programa principal

Por otro lado, se ha hecho un control exhaustivo de los parámetros de entrada del programa principal. Esto es dado que para los datos hemos empleado long sin signo, en caso de recibir números negativos, nuestro programa no rechaza debido a la imposibilidad de crear un número de hilos negativos, sino que se castea LONG_MAX - 2, por lo que trataría de crear 18 trillones de hilos. Por esto hacemos un control de los parámetros de entrada:

```
danyyil at debian13 in ~/INFORMATICA/2º/SO/P1/Miner on
∴ ./mrush
Miner exited unexpectedly!
Usage: ./miner <TARGET_INI> <ROUNDS> <N_THREADS>
danyyil at debian13 in ~/INFORMATICA/2º/SO/P1/Miner on
∴ ./mrush 0
Miner exited unexpectedly!
Usage: ./miner <TARGET_INI> <ROUNDS> <N_THREADS>
danyyil at debian13 in ~/INFORMATICA/2º/SO/P1/Miner on
∴ ./mrush 0 -2 1
Target, rounds and number of threads MUST be positive

danyyil at debian13 in ~/INFORMATICA/2º/SO/P1/Miner on
∴ ./mrush 0 5 -1
Target, rounds and number of threads MUST be positive

danyyil at debian13 in ~/INFORMATICA/2º/SO/P1/Miner on
∴ ./mrush -1 5 1
Target, rounds and number of threads MUST be positive
```

Figura 2: Resultados al introducir parámetros inválidos

3.4. Pruebas unitarias

Para la comprobación de ciertas funciones, se han realizado pruebas unitarias. Hemos tomado el formato y ficheros que se nos proporcionó en la asignatura de Proyecto de Programación de 1º y lo hemos adaptado a este proyecto. Hay funciones que no tienen pruebas unitarias debido a que el funcionamiento de estas es completamente regular y no tienen casos diferentes de funcionamiento, como podría ser el registrador, que solamente registra información. Estas funciones que no tienen pruebas unitarias, hacen el control de parámetros mediante asserts.

```
└─ .:: ./script-test.sh 0
Ejecutando TODOS los tests...
Entrando en el directorio: miner_test
Compilando en miner_test...
gcc -c -Wall -Wextra -std=gnu99 -pedantic -g miner_test.c
gcc -c -Wall -Wextra -std=gnu99 -pedantic -g miner.c
gcc -c -Wall -Wextra -std=gnu99 -pedantic -g logger.c
gcc -c -Wall -Wextra -std=gnu99 -pedantic -g pow.c
gcc -o miner_test miner_test.o miner.o logger.o pow.o
Ejecutando miner_test en modo normal...
Running all test for module Miner:
miner_test.c line 66 test1_open_pipes: PASS
miner_test.c line 72 test2_open_pipes: PASS
Solution accepted: 00024849 --> 00000000
miner_test.c line 84 test1_pow_seek: PASS
miner_test.c line 98 test2_pow_seek: PASS
miner_test.c line 112 test3_pow_seek: PASS
Tests passed 100%
Limpiando miner_test...
Entrando en el directorio: pow_test
Compilando en pow_test...
cc -c -Wall -Wextra -std=gnu99 -pedantic -g pow_test.c
pow_test.c: In function 'test3_pow':
pow_test.c:71:57: warning: integer constant is so large that it is unsigned
   71 |   PRINT_TEST_RESULT(fx == pow_hash(18446744073709551615));
      |                                                         ^
cc -c -Wall -Wextra -std=gnu99 -pedantic -g pow.c
cc -o pow_test *.o
Ejecutando pow_test en modo normal...
Running all test for module Pow:
pow_test.c line 59 test1_pow: PASS
pow_test.c line 65 test2_pow: PASS
pow_test.c line 71 test3_pow: PASS
Tests passed 100%
Limpiando pow_test...
```

Figura 3: Traza de ejecución de tests unitarios

3.5. Pruebas de memoria

Tanto en las pruebas unitarias como en el programa principal, en los Makefile se ha añadido la opción de ejecutar con valgrind para revisar posibles fugas de memoria

```
# Comando ejecutado con make valgrind
$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes --log-file=
    memcheck.log ./$(EXE) 1 10 8


==40371== Memcheck, a memory error detector
==40371== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==40371== Using Valgrind-3.24.0 and LibVEX; rerun with -h for copyright info
==40371== Command: ./mrush 1 10 8
==40371== Parent PID: 40353
==40371==
==40372==
==40372== HEAP SUMMARY:
==40372==     in use at exit: 0 bytes in 0 blocks
==40372==   total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
==40372==
==40372== All heap blocks were freed -- no leaks are possible
==40372==
==40372== For lists of detected and suppressed errors, rerun with: -s
==40372== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
==40371==
==40371== HEAP SUMMARY:
==40371==     in use at exit: 0 bytes in 0 blocks
==40371==   total heap usage: 103 allocs, 103 frees, 16,496 bytes allocated
==40371==
==40371== All heap blocks were freed -- no leaks are possible
==40371==
==40371== For lists of detected and suppressed errors, rerun with: -s
==40371== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

3.6. Pruebas de sincronización de hilos

Además de la comprobación de fugas de memoria, valgrind tiene la herramiento helgrind, que nos informa de posibles accesos concurrentes a datos y condiciones de carrera. Dado que en este primer apartado no hemos implementado exclusión mutua ni usado semáforos, esta herramienta nos da alertas, pero la hemos decidido dejar para la posteridad. Dado que el log es muy extenso, enseñamos una pequeña parte de él, pero en el proyecto se redirige la salida a un archivo con el registro completo.

```
$ valgrind --tool=helgrind --log-file=thread_check.log ./mrush 1 5 4

==43214== Helgrind, a thread error detector
==43214== Copyright (C) 2007-2024, and GNU GPL'd, by OpenWorks LLP et al.
==43214== Using Valgrind-3.24.0 and LibVEX; rerun with -h for copyright info
==43214== Command: ./mrush 1 5 4
==43214== Parent PID: 43189
==43214==
==43214== ---Thread-Announcement------------------------------------------
==43214==
==43214== Thread #3 was created
==43214==    at 0x49885DF: clone (clone.S:76)
==43214==    by 0x4988730: __clone_internal_fallback (clone-internal.c:64)
==43214==    by 0x4988730: __clone_internal (clone-internal.c:109)
==43214==    by 0x490A71F: create_thread (pthread_create.c:298)
==43214==    by 0x490B2CE: pthread_create@@GLIBC_2.34 (pthread_create.c:841)
==43214==    by 0x4853285: pthread_create_WRK (hg_intercepts.c:445)
==43214==    by 0x109938: calcular_solucion (miner.c:112)
==43214==    by 0x109AF3: minero (miner.c:154)
==43214==    by 0x109DC1: main (mrush.c:64)
==43214==
==43214== Possible data race during read of size 4 at 0x1FFEFFF9BC by thread #3
==43214== Locks held: none
==43214==    at 0x1096A7: pow_seek (miner.c:59)
==43214==    by 0x485348A: mythread_wrapper (hg_intercepts.c:406)
==43214==    by 0x490AB7A: start_thread (pthread_create.c:448)
==43214==    by 0x49885EF: clone (clone.S:100)
==43214==
==43214== This conflicts with a previous write of size 4 by thread #2
==43214== Locks held: none
==43214==    at 0x10972E: pow_seek (miner.c:73)
==43214==    by 0x485348A: mythread_wrapper (hg_intercepts.c:406)
==43214==    by 0x490AB7A: start_thread (pthread_create.c:448)
==43214==    by 0x49885EF: clone (clone.S:100)
==43214==  Address 0x1ffefff9bc is on thread #1's stack
==43214==  in frame #5, created by calcular_solucion (miner.c:84)
==43241==
==43241== Use --history-level=approx or =none to gain increased speed, at
==43241== the cost of reduced accuracy of conflicting-access information
==43241== For lists of detected and suppressed errors, rerun with: -s
==43241== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
==43214==
==43214== Use --history-level=approx or =none to gain increased speed, at
==43214== the cost of reduced accuracy of conflicting-access information
==43214== For lists of detected and suppressed errors, rerun with: -s
==43214== ERROR SUMMARY: 13 errors from 2 contexts (suppressed: 3 from 1)
```

3.7. Pruebas de razonamiento y resultados

Por último, hemos implementado un makefile aparte del principal, que se encarga de hacer pruebas del programa, variando tanto número de rondas como número de hilos creados. Hemos medido los resultados con la herramienta time de la shell zsh. Esta herramienta se diferencia del time de bash, en que además de los tiempos reales de usuarios y sistema, nos añade un porcentaje de utilización de CPU, que nos es muy útil para poder aproximar cuánto se han aprovechado las CPUs. Por ejemplo, para la variación de hilos:

```
TARGET_INI = 0
ROUNDS_FIJAS = 20
THREADS_FIJOS = 4

.PHONY: test_threads test_rounds test_all

# Prueba variando hilos
test_threads: $(EXE)
  @mkdir -p test_logs
  @echo ""
  @echo "=========================================================="
  @echo " PRUEBA 1: Variando numero de hilos (Rondas fijas: $(ROUNDS_FIJAS))"
  @echo "=========================================================="
  @echo ""
  @echo "---> Ejecutando con 1 HILO"
  @cd test_logs && time ../$(EXE) $(TARGET_INI) $(ROUNDS_FIJAS) 1
  @echo ""
  @echo "---> Ejecutando con 2 HILOS"
  @cd test_logs && time ../$(EXE) $(TARGET_INI) $(ROUNDS_FIJAS) 2
  @echo ""
  @echo "---> Ejecutando con 4 HILOS"
  @cd test_logs && time ../$(EXE) $(TARGET_INI) $(ROUNDS_FIJAS) 4
  @echo ""
  @echo "---> Ejecutando con 8 HILOS"
  @cd test_logs && time ../$(EXE) $(TARGET_INI) $(ROUNDS_FIJAS) 8
  @echo ""
  @echo "---> Ejecutando con 16 HILOS"
  @cd test_logs && time ../$(EXE) $(TARGET_INI) $(ROUNDS_FIJAS) 16
  @echo ""
  @echo "---> Ejecutando con 32 HILOS"
  @cd test_logs && time ../$(EXE) $(TARGET_INI) $(ROUNDS_FIJAS) 32
  @echo ""
  @echo "---> Ejecutando con 64 HILOS"
  @cd test_logs && time ../$(EXE) $(TARGET_INI) $(ROUNDS_FIJAS) 64
```

4. FUNCIONALIDADES ADICIONALES

4.1. Automatización de tests de rendimiento

Además de los tests solicitados, hemos creado un framework de pruebas, que redirige su registro a una carpeta en la que tenemos toda la información del registrador, además de informarnos de los rendimientos para distintos casos de uso. Con los comandos make test_threads, make test_rounds y make test_all automatizamos la ejecución del benchmarking.

4.2. Alineación de structs

Nos hemos encargado de alinear las estructuras empleadas a múltiplos de 8 bytes, puesto que los procesadores empleados son de 64 bit. Gracias a esto, optimizamos el rendimiento de nuestro programa facilitando al compilador la alineación de memoria y acelerando la transmisión de datos en las tuberías.

4.3. Documentación

Hemos empleado Doxygen e instrucciones de Makefile para la generación automática de un documento pdf con registro de la documentación completa del programa, encadenando la generación de la documentación, con la compilación del motor LaTeX para generar el documento final.

4.4. Macros personalizadas para legibilidad

Además de la creación de macros para gestión de errores como hemos mencionado antes, hemos creado un fichero types.h con tipos propios definidos que facilitan la programación y mejoran la legibilidad del código

```
typedef int32_t i32;  /**< Entero con signo 32 bit. [-2^31, 2^31 - 1] */
typedef int64_t i64;  /**< Entero con signo 64 bit. [-2^63, 2^63 - 1] */
typedef uint32_t u32; /**< Entero sin signo 32 bit. [0, 2^32 - 1] */
typedef uint64_t u64; /**< Entero sin signo 64 bit. [0, 2^64 - 1] */
typedef float f32;    /**< Float 32 bit. 7 dig. precision */
typedef double f64;   /**< Float 64 bit. 15 dig. precision */

#define READ 0  /**< Indice del descriptor de archivo para lectura */
#define WRITE 1 /**< Indice del descriptor de archivo para escritura */

#define FOUND 1 /**< Indica que un hilo ha encontrado solucion */

#define MIN(a, b) ((a) < (b) ? (a) : (b))
```

5. CUESTIONES

Se pide probar el programa para diferentes números de rondas e hilos, buscando tomar alguna conclusión sobre el número óptimo de hilos.

En primer lugar, tomamos las especificaciones del sistema de pruebas.

- SO: Debian GNU/Linux 13 x86_64
- CPU: AMD Ryzen 7 4800H (16) @ 2.90 GHz

```
CPU NODE SOCKET CORE L1d:L1i:L2:L3 ONLINE    MAXMHZ    MINMHZ       MHZ
  0    0      0    0 0:0:0:0          yes 2900.0000 1400.0000 1842.5580
  1    0      0    0 0:0:0:0          yes 2900.0000 1400.0000 1673.8621
  2    0      0    1 1:1:1:0          yes 2900.0000 1400.0000 1302.5081
  3    0      0    1 1:1:1:0          yes 2900.0000 1400.0000 1490.7560
  4    0      0    2 2:2:2:0          yes 2900.0000 1400.0000 1772.1570
  5    0      0    2 2:2:2:0          yes 2900.0000 1400.0000 1785.5500
  6    0      0    3 3:3:3:0          yes 2900.0000 1400.0000 2289.3379
  7    0      0    3 3:3:3:0          yes 2900.0000 1400.0000 2436.1509
  8    0      0    4 4:4:4:1          yes 2900.0000 1400.0000 4251.9312
  9    0      0    4 4:4:4:1          yes 2900.0000 1400.0000 4256.5820
 10    0      0    5 5:5:5:1          yes 2900.0000 1400.0000 2718.2910
 11    0      0    5 5:5:5:1          yes 2900.0000 1400.0000 3277.1470
 12    0      0    6 6:6:6:1          yes 2900.0000 1400.0000 3215.0271
 13    0      0    6 6:6:6:1          yes 2900.0000 1400.0000 2576.7639
 14    0      0    7 7:7:7:1          yes 2900.0000 1400.0000 1907.2090
 15    0      0    7 7:7:7:1          yes 2900.0000 1400.0000 1897.8510
```

Trabajamos en un portátil con 16 núcleos, dato que nos va a ser relevante a la hora de hacer las pruebas.

Las pruebas que vamos a realizar serán variando las rondas entre 20, 40 y 100. Dentro de cada una de esas, realizaremos pruebas para número de hilos 1, 2, 4, 8, 16, 32 y 64.

Antes de comenzar vemos el trabajo inicial de las CPUs, para poder hacernos una idea del trabajo máximo posible. Al tener 16 CPUs, el máximo sería 1600 %, pero todas las CPUs se están usando en ~20 %, luego el máximo disponible en teoría debería ser ~1280 %

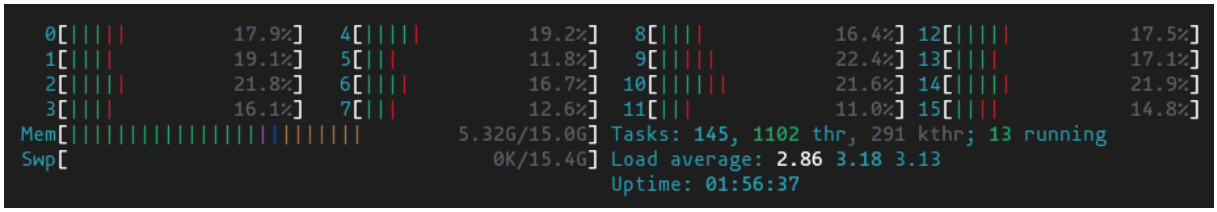

Figura 4: Trabajo inicial CPUs

5.1. Ejecución para 20, 40 y 100 rondas

Los resultados de la función time son:

```
# 20 RONDAS
../mrush 0 20 1  0.28s user 0.00s system 99% cpu 0.287 total
../mrush 0 20 2  0.35s user 0.01s system 186% cpu 0.196 total
../mrush 0 20 4  0.29s user 0.00s system 381% cpu 0.076 total
../mrush 0 20 8  0.43s user 0.00s system 621% cpu 0.069 total
../mrush 0 20 16  0.45s user 0.02s system 1045% cpu 0.045 total
../mrush 0 20 32  0.46s user 0.03s system 953% cpu 0.052 total
../mrush 0 20 64  0.45s user 0.08s system 698% cpu 0.077 total

# 40 RONDAS
../mrush 0 40 1  0.62s user 0.01s system 99% cpu 0.633 total
../mrush 0 40 2  0.77s user 0.00s system 180% cpu 0.432 total
../mrush 0 40 4  0.68s user 0.02s system 362% cpu 0.193 total
../mrush 0 40 8  0.67s user 0.02s system 655% cpu 0.104 total
../mrush 0 40 16  0.97s user 0.04s system 1037% cpu 0.097 total
../mrush 0 40 32  1.05s user 0.10s system 906% cpu 0.126 total
../mrush 0 40 64  0.90s user 0.17s system 684% cpu 0.156 total

# 100 RONDAS
../mrush 0 100 1  1.66s user 0.02s system 99% cpu 1.684 total
../mrush 0 100 2  1.83s user 0.02s system 185% cpu 0.997 total
../mrush 0 100 4  1.74s user 0.02s system 359% cpu 0.490 total
../mrush 0 100 8  1.43s user 0.03s system 697% cpu 0.209 total
../mrush 0 100 16  1.97s user 0.15s system 1025% cpu 0.207 total
../mrush 0 100 32  2.21s user 0.26s system 927% cpu 0.265 total
../mrush 0 100 64  2.03s user 0.49s system 685% cpu 0.367 total
```

De estos datos tomaremos el tiempo en modo usuario, el tiempo de ejecución total, y el porcentaje de uso de CPU y los representaremos frente al número de rondas. Obteniendo así las siguientes gráficas:

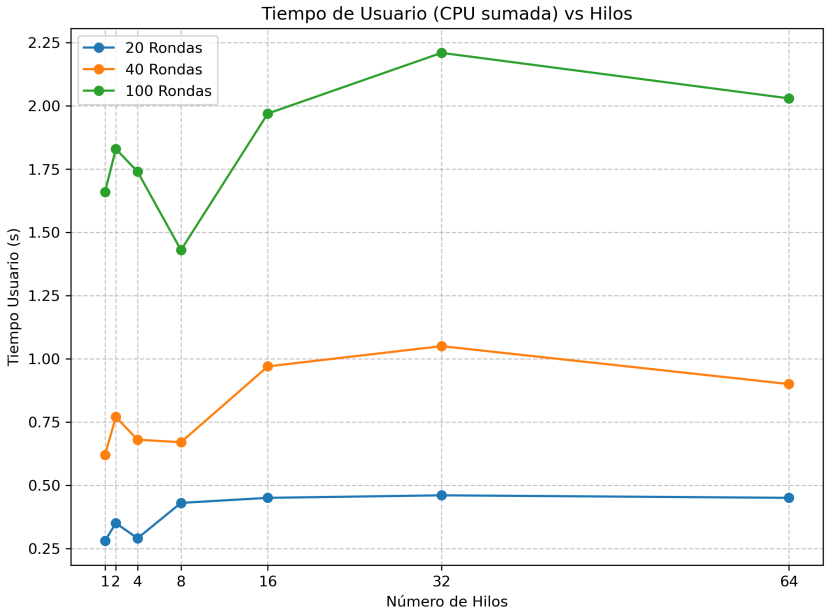

Figura 5: Tiempo en modo usuario. 20, 40 y 100 rondas

A diferencia del tiempo total, el tiempo de usuario crece constantemente. Esto demuestra que añadir más hilos siempre introduce un overhead (coste extra de gestión, creación de hilos y sincronización por parte del SO). Aunque el usuario espere menos frente a la pantalla, la máquina está trabajando mucho más.

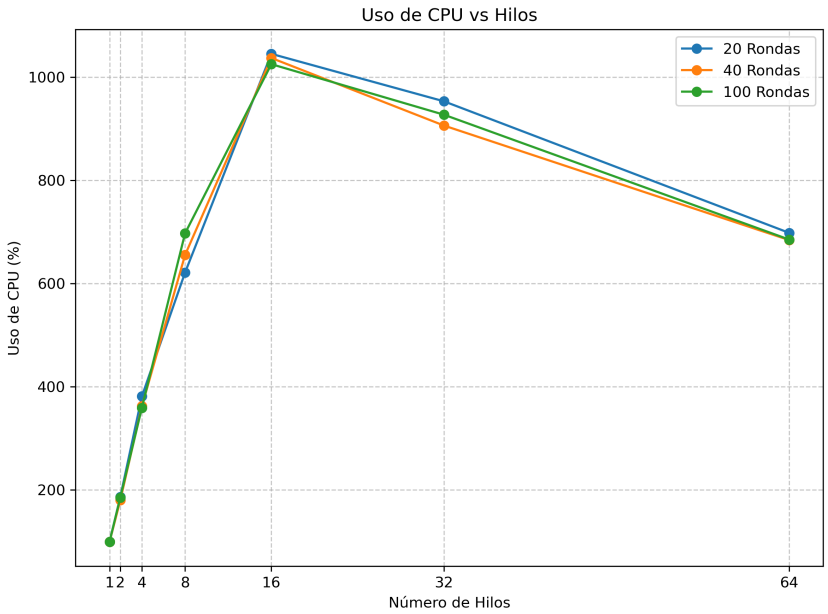

Figura 6: Porcentaje de uso CPU. 20, 40 y 100 rondas

En cuanto al porcentaje de uso de la CPU, vemos que crece de forma casi lineal hasta los 16 hilos (alcanzando más de un 1000 %, es decir, saturando 10 núcleos). Curiosamente, al llegar a 32 y 64 hilos el uso global de CPU cae. Al haber demasiados hilos peleando por los recursos, estos pasan gran parte

de su ciclo vital bloqueados o en las colas del planificador en lugar de haciendo cálculos.

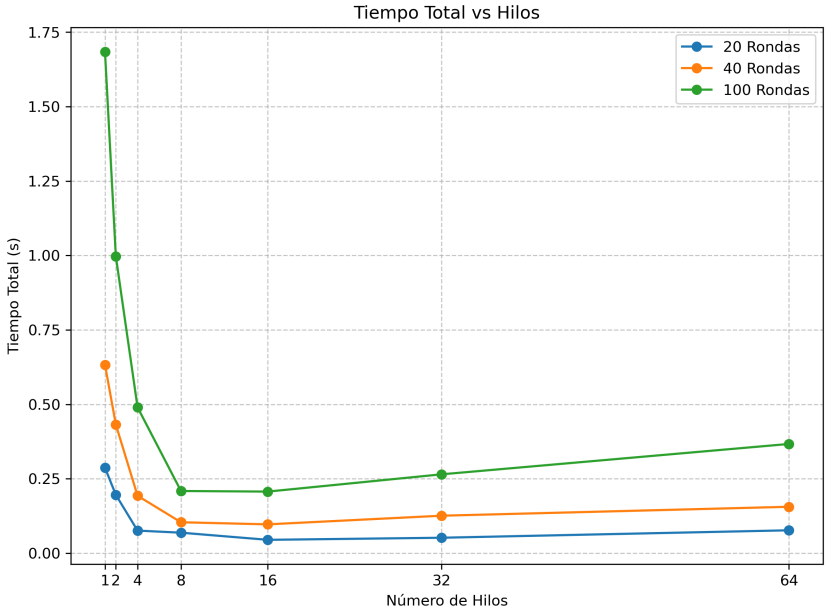

Figura 7: Tiempo total de ejecucion. 20, 40 y 100 rondas

Se observa una mejora drástica del rendimiento al paralelizar el problema pasando de 1 a 8-16 hilos. Sin embargo, a partir de los 16 hilos la curva se aplana e incluso repunta (el programa tarda más con 32 o 64 hilos). Esto ocurre porque el sistema operativo gasta más tiempo gestionando el cambio de contexto (context switch) entre tantísimos hilos que ejecutando el código matemático real.

5.2. Búsqueda del número óptimo de hilos

Tras todo esto, vemos que el número óptimo de hilos en nuestro caso particular debe estar entre 8 y 16 hilos. Por lo que nuevamente realizamos las pruebas, variando únicamente entre 8 y 16 hilos.

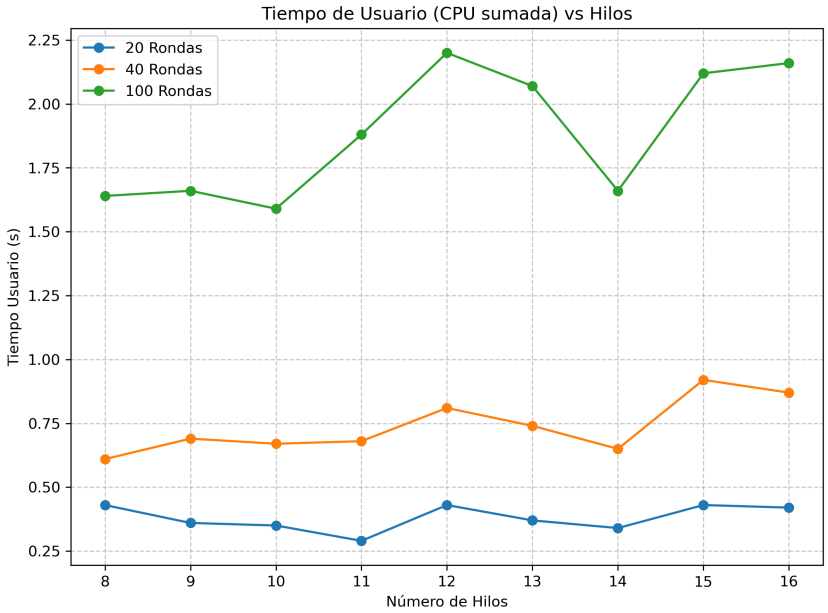

Figura 8: Tiempo en modo usuario. 20, 40, 100 rondas. Hilos 8-16

La gráfica de tiempo en modo usuario muestra cierta volatilidad (picos en 12 y 15 hilos). Esto refleja la aleatoriedad natural del sistema operativo al asignar los tiempos de procesador y las pequeñas colisiones de memoria caché cuando demasiados hilos compiten fuertemente.

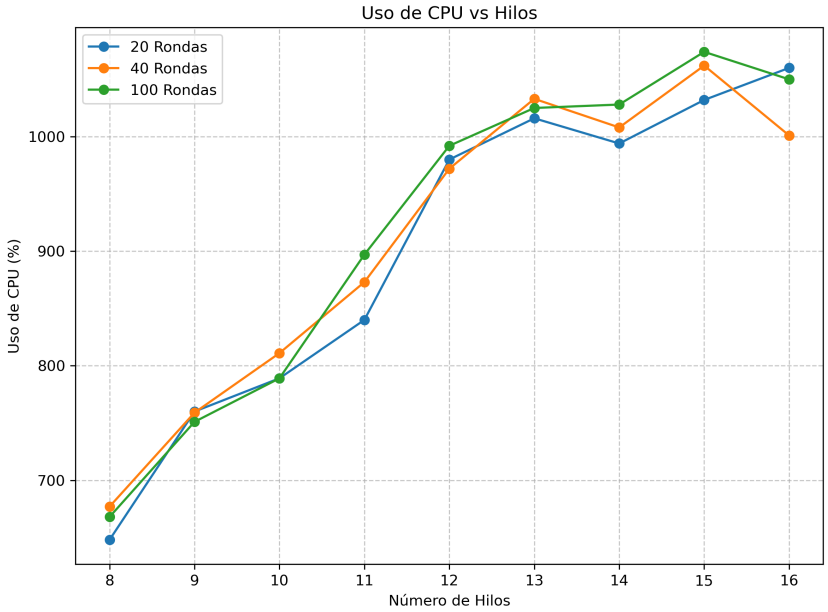

Figura 9: Porcentaje de uso de CPU. 20, 40, 100 rondas. Hilos 8-16

La escala muestra un aprovechamiento brutal del hardware de la máquina, escalando desde un 701 % (usando unos 7 núcleos a tope) en 8 hilos, hasta estabilizarse sobrepasando el 1000 % entre 14 y 16 hilos.

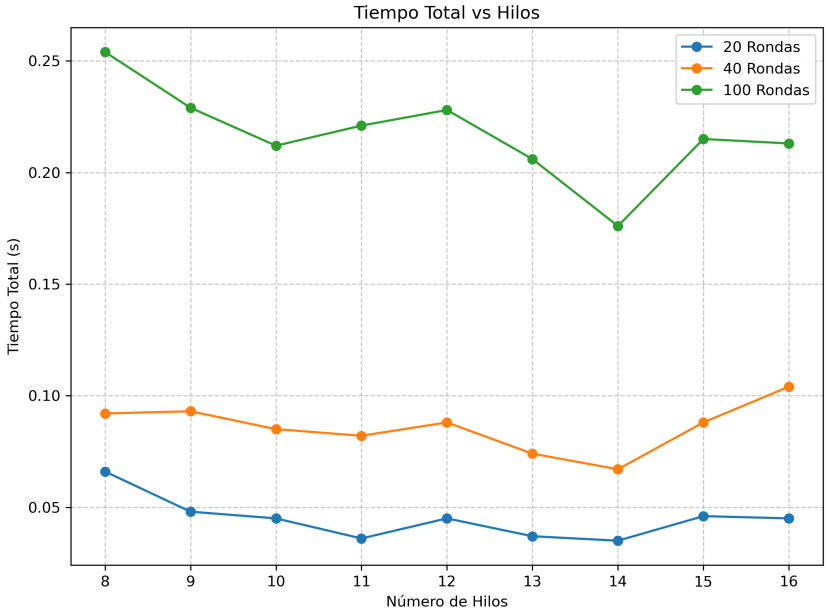

Figura 10: Detalle del tiempo de ejecución total en el rango óptimo (8-16 hilos).

Al aislar el rango óptimo observado anteriormente, descubrimos que el rendimiento máximo absoluto (el tiempo más bajo) se alcanza exactamente a los 14 hilos (0.033 segundos). A partir de los 15 y 16 hilos, el tiempo total comienza a empeorar ligeramente por la saturación del planificador de procesos.

6. Conclusiones

En resumen, hemos podido aprender a crear y manejar procesos, evitando huérfanos y zombies. A su vez hemos podido ver y medir la eficiencia de la división de carga de trabajo en hilos. Además, hemos conseguido implementar un IPC funcional entre dos procesos dependientes el uno del otro de manera segura, controlada y eficiente. Por último, hemos realizado un benchmarking exhaustivo de nuestro programa en una máquina específica, y hemos podido medir el valor óptimo de hilos de trabajo.

7. ANEXO

Una modificación importante que hemos realizado ha sido sobre la función pow_hash. En principio, la función tomaba y devolvía datos de tipo long int, es decir, entero con signo de 8 bytes. Sin embargo, el enunciado especifica que cuando hagamos la búsqueda de las soluciones del POW, se haga solamente para un rango de valores positivo. Teniendo en cuenta que el operador módulo de C, no es el módulo matemático y es capaz de devolver valores negativos, hemos refactorizado la función:

```
u64 pow_hash(u64 x) {
  u64 result = (x * BIG_X + BIG_Y) % PRIME;
  return result;
}
```

Por otro lado, añadimos los enlaces adicionales que han ayudado en el proyecto, que no son bibliografía como tal:

- Funciones macros en C
- Documentación con Doxygen
- Inline en C
  En principio, pensamos en hacer que die fuera estática e inline, debido a las numerosas llamadas, pero al ser realmente una función de terminación del programa, solo se ejecutaría una vez, por lo que decidimos convertirla en una macro.